

UNIT-3

UNIT - III:

Introduction to Python programming, Introduction to RaspberryPi, Interfacing Raspberry Pi with basic peripherals, Implementation of IoT with RaspberryPi.

1.Introduction to Python Programming

Introduction to Python Programming

Python is a **high-level, interpreted, general-purpose programming language** developed by **Guido van Rossum** and released in **1991**.

Applications of Python

Python is widely used for:

- Web development (e.g., using Django, Flask)
 - Software development & scripting
 - System administration
 - Mathematics and Data Science
 - Internet of Things (IoT)
 - Artificial Intelligence and Machine Learning
 - Database operations
 - Big data handling and analytics
-

Why Python?

Python has become one of the most popular programming languages due to the following reasons:

1. **Cross-Platform Support:** Runs on Windows, macOS, Linux, Raspberry Pi, etc.
 2. **Simple Syntax:** Close to English and very readable.
 3. **Fewer Lines of Code:** Achieve more with less.
 4. **Interpreted Language:** No need to compile; run code directly.
 5. **Multi-paradigm:** Supports procedural, object-oriented, and functional programming.
-

★ Features of Python

1. Simple and Easy to Learn

- Python syntax is minimalistic and easily readable.
- Example:

```
python
CopyEdit
print("Hello, world!")
```

2. Free and Open Source

- Freely available and supported by a strong community.

3. High-Level Language

- You don't need to manage memory or deal with low-level operations.

4. Portable

- Write once, run anywhere (if you avoid system-dependent features).

5. Object-Oriented

- Supports OOP concepts like classes, inheritance, encapsulation.

6. Interpreted

- Runs line by line using the Python interpreter (no compilation needed).

□ Standard Data Types in Python

🔢 Numbers:

- **Types:** int, float, complex
- Example:

```
python
Copy Edit

a = 5
print(type(a)) # int

b = 40.5
print(type(b)) # float

c = 1 + 3j
print(isinstance(c, complex)) # True
```

🔤 Strings:

- Defined using ' ', " ", or ''' ''' (for multi-line)
- Supports slicing, concatenation, repetition
- Example:

```
str1 = "Hello"
str2 = "World"
print(str1 + " " + str2) # Hello World
print(str1[0:2]) # He
print(str1 * 2) # HelloHello
```

Lists:

- Mutable (can change values)
- Enclosed in square brackets []
- Example:

```
python

list1 = [1, "Python", 3.5]
print(list1[1]) # Python
print(list1[0:2]) # [1, "Python"]
print(list1 * 2) # Repetition
print(list1 + list1) # Concatenation
```

Tuples:

- Immutable (cannot change after creation)
- Enclosed in parentheses ()
- Example:

```
python

tup = (1, "Python", 3.5)
print(tup[1:]) # ("Python", 3.5)
print(tup * 2)
```

Dictionaries:

- Key-value pairs, unordered
- Enclosed in curly braces {} with a colon :
- Example:

```
d = {1: 'Alice', 2: 'Bob'}
print(d[1])          # Alice
print(d.keys())       # dict_keys([1, 2])
print(d.values())     # dict_values(['Alice', 'Bob'])
```

□ Python Functions

A function is a block of code designed to perform a specific task.

► Defining a Function:

```
python
CopyEdit
def greet():
    print("Hello")
```

► Calling a Function:

```
python
CopyEdit
greet() # Output: Hello
```

📌 Arguments in Functions

• Positional Arguments:

```
python
Copy Edit

def full_name(fname, lname):
    print(fname + " " + lname)
full_name("Shiva", "Kumar")
```

• Arbitrary Arguments (*args):

```
python
Copy Edit

def my_function(*kids):
    print("The youngest is", kids[2])
my_function("Ravi", "Sita", "Geeta")
```

• Return Values:

```
python
Copy Edit

def multiply(x):
    return 5 * x

print(multiply(3)) # 15
```

2.Introduction to Raspberry pie

✓ Introduction to Raspberry Pi (Short Notes)

- **Raspberry Pi** is a small, affordable **single-board computer** developed by the Raspberry Pi Foundation.
- It is used for learning programming, electronics, **IoT**, robotics, and building DIY projects.

Today, Raspberry Pi is widely used in:

- **Hobby electronics**
 - **IoT projects**
 - **Robotics**
 - **DIY automation**
 - **Prototyping**
 - **Media servers**
-

History

- Developed in 2006 by **Eben Upton** to teach computer science in schools.
 - First released in **2012**.
-

Hardware Features

- ARM Processor, GPU, RAM
 - USB, HDMI, Audio Jack, Ethernet
 - **GPIO Pins** to connect sensors and devices
 - SD card for OS and storage
-

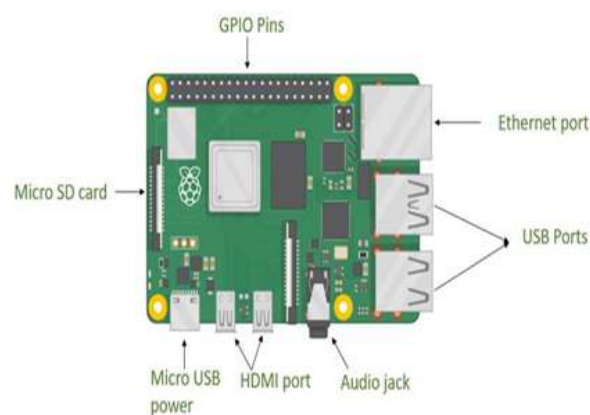
Operating Systems

- **Raspberry Pi OS** (official)
- Ubuntu, Windows 10 IoT Core, and others

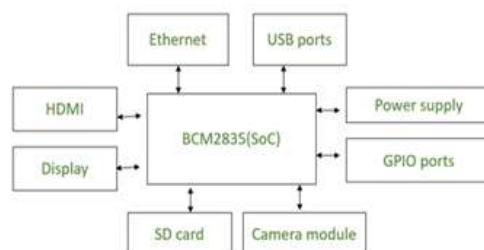
Architecture of Raspberry Pi

Raspberry Pi is a small single-board computer (SBC). It is a credit card-sized computer that can be plugged into a monitor. It acts as a minicomputer by connecting the keyboard, mouse, and display. Raspberry Pi has an ARM processor and 512MB of RAM. The architecture of Raspberry Pi is discussed in this article.

The following diagram shows the architecture of Raspberry Pi:



The following diagram shows some main blocks of Raspberry Pi:



🔗 Applications

- Programming and education
 - Home automation
 - IoT and sensor-based projects
 - Media center (Kodi)
 - Retro gaming (RetroPie)
-

🆚 Raspberry Pi vs Arduino

Feature	Raspberry Pi	Arduino
Type	Single-board computer (SBC)	Microcontroller board
OS	Runs full OS like Linux	No OS (bare-metal programming)
Programming	Python, Java, C++, etc.	Mostly C/C++ via Arduino IDE
Use case	General computing + hardware control	Precise control of sensors/actuators
Power	Requires stable 5V supply	Runs on USB or 9V battery
Cost	Slightly higher	Generally cheaper

3. Interfacing Raspberry pi with basic peripheral

🔌 Interfacing Raspberry Pi with Basic Peripherals (Detailed Explanation)

✓ What is Interfacing?

Interfacing means connecting **external devices** (called peripherals) like:

- Keyboard, mouse
 - Monitor
 - LED, sensor
 - Camera, speaker
- to the Raspberry Pi so they can **communicate and interact** with each other.
-

📦 Required Materials:

- Raspberry Pi board (any model)
 - SD card (with Raspberry Pi OS)
 - Power adapter (5V)
 - Monitor + HDMI cable
 - USB keyboard and mouse
 - Breadboard + jumper wires (for sensors/LEDs)
 - Internet connection (Wi-Fi or Ethernet)
-

🔧 Steps for Interfacing Basic Peripherals

1. 📄 Boot Up the Raspberry Pi

- Insert **microSD card** with Raspberry Pi OS installed
 - Connect:
 - HDMI cable → Monitor
 - USB keyboard and mouse
 - Power ON the Raspberry Pi using a **5V micro-USB/USB-C adapter**
 - Raspberry Pi boots and shows GUI/Desktop on the monitor
-

2. 🖱️ USB Devices (Plug and Play)

- USB devices such as:
 - Keyboard
 - Mouse
 - USB pendrive
- Are automatically detected when connected to USB ports.

☞ No drivers or configuration needed for basic input/output devices.

3. 🧠 Interfacing GPIO (General Purpose Input/Output) Devices

Raspberry Pi has **40 GPIO pins** used to control or receive signals from devices like:

- LED (output)
- Button (input)
- Sensors (e.g., DHT11)
- Motors, buzzers

🔗 *Example: LED Blinking Using GPIO*

★ Python Code

```
import RPi.GPIO as GPIO
import time

GPIO.setmode(GPIO.BCM)
GPIO.setup(17, GPIO.OUT)

try:
    while True:
        GPIO.output(17, True)
        time.sleep(1)
        GPIO.output(17, False)
        time.sleep(1)
except KeyboardInterrupt:
    GPIO.cleanup()
```

🔗 Explanation:

- GPIO 17 is set as output
 - Turns LED ON for 1 second, OFF for 1 second in a loop
-

4. 📷 Camera Interface (CSI Port)

- Connect **Pi Camera** using CSI ribbon cable
 - Enable camera from `raspi-config` → Interface Options → Enable Camera
 - Can be used for:
 - Image capture
 - Video streaming
 - Face detection using OpenCV
-

5. 🖥️ LCD Display Interface (DSI Port)

- Connect official Raspberry Pi touchscreen
 - Useful for making tablets, control panels, IoT dashboards
-

6. 📡 Using I2C and SPI Devices

📡 I2C (Inter-Integrated Circuit)

- Used for communication with sensors (e.g., temperature, light, pressure)
- Only **2 wires**: SDA (Data), SCL (Clock)
- Enable via `raspi-config`

📡 SPI (Serial Peripheral Interface)

- Faster than I2C
 - Connects devices like displays, RFID readers
 - Pins used: MISO, MOSI, SCLK, CE0/CE1
-

7. 🎧 Audio and Video Output

- **Audio**: 3.5 mm audio jack or HDMI (for sound output)
 - **Video**: HDMI port supports HD output to monitors/TVs
-

8. 🌐 Networking Peripherals

- **Ethernet port** or **Wi-Fi** for internet connection
- Enables Raspberry Pi to:
 - Access the web
 - Host servers
 - Send sensor data to the cloud

4. Implementation of IOT with Raspberry Pi

Implementation of IoT with Raspberry Pi (Detailed Explanation)

✔ What is IoT with Raspberry Pi?

IoT (Internet of Things) involves connecting sensors, devices, and systems to the internet to **collect, analyze, and act on data**.

Raspberry Pi acts as the **central controller (edge device)** to:

- Read data from sensors
 - Process it using Python
 - Control actuators (like LEDs or motors)
 - Send data to the **cloud** or provide a **web interface** for monitoring
-

❑ Components Required

Component	Description
Raspberry Pi	Acts as the IoT controller
DHT11/DHT22 Sensor	Measures temperature and humidity
LED	Acts as an output/alert indicator
Breadboard	To easily connect components
Jumper Wires	For wiring
Power Supply	5V to power Raspberry Pi
Internet	For cloud connectivity (via Wi-Fi/Ethernet)

Step-by-Step Implementation

◆ Step 1: Setup Hardware

Connect DHT11 Sensor to GPIO:

- VCC → 3.3V pin
- GND → Ground pin
- DATA → GPIO pin 4 (you can change this in code)

Connect LED:

- Anode (longer leg) → 220Ω resistor → GPIO pin 17
 - Cathode (shorter leg) → Ground
-

◆ Step 2: Install Required Libraries

In terminal, run:

```
bash
CopyEdit
sudo apt update
sudo apt upgrade
sudo pip3 install Adafruit_DHT
```

This installs the Adafruit library to interact with the DHT sensor.

◆ Step 3: Write Python Code

Create a file `iot_example.py`:

What This Code Does:

- Reads **temperature & humidity**
- If temperature > 25°C → **turns LED ON**
- Else → **turns LED OFF**
- Repeats every 2 seconds

```

import Adafruit_DHT
import RPi.GPIO as GPIO
import time

# Use BCM pin numbering
GPIO.setmode(GPIO.BCM)

GPIO.setup(17, GPIO.OUT) # LED on GPIO 17
sensor = Adafruit_DHT.DHT11
pin = 4 # DHT11 data pin

try:
    while True:
        # Read from DHT11 sensor
        humidity, temperature = Adafruit_DHT.read_retry(sensor, pin)

        # If data is valid
        if temperature is not None:
            print("Temperature:", temperature, "°C")
            print("Humidity:", humidity, "%")

            if temperature > 25:
                GPIO.output(17, True) # Turn LED ON
            else:
                GPIO.output(17, False) # Turn LED OFF
        else:
            print("Failed to read from sensor")

        time.sleep(2)

except KeyboardInterrupt:
    GPIO.cleanup()

```

◆ Step 4: Run the Script

In terminal:

```

bash
CopyEdit
python3 iot_example.py

```

You will see:

- **Temperature and humidity readings printed every 2 seconds**
 - **LED toggles ON/OFF** based on temperature ($>25^{\circ}\text{C}$)
-

Expanding the IoT Project

1. Data Logging

- Store sensor readings in a `.csv` file for analysis

2. Cloud Connectivity

- Use MQTT, Firebase, or Thingspeak to:
 - Upload sensor data to the cloud
 - View data from anywhere

3. Remote Access with Flask

- Create a Python **Flask web app** to show sensor data and control devices from a browser

4. Automation with IFTTT

- Send alerts via email or phone if temperature exceeds limit
-

Benefits of Using Raspberry Pi for IoT

Feature	Benefit
GPIO Pins	Interface with real-world devices
Python Support	Easy coding and vast library support
Internet-Enabled	Supports Wi-Fi, Ethernet for cloud use
Linux OS	Full OS allows multitasking and servers
Cheap & Portable	Ideal for small IoT solutions

Example Applications

- **Smart home system** (light/fan automation)

- **Weather station** (temperature/humidity monitoring)
- **Remote sensor** with cloud reporting
- **Home security alert system**